

Design Patterns Final Project Guidelines

The final project for the **Design Patterns Lab** will be completed by teams of **two students**. The primary objective of the project is not simply to develop a functional software application, but to demonstrate the ability to analyze a software problem, design an appropriate solution, apply suitable design patterns, and develop a maintainable and extensible application. Each team will have the freedom to select its own application domain and problem, provided that the selected project satisfies the scope and technical requirements specified below. The quality of the project will be evaluated primarily on the quality of its design, meaningful application of design patterns, implementation practices, database design, maintainability, and the ability of the team to justify its design decisions.

Each team must develop a **desktop-based application using JavaFX**, with **Maven** used for project configuration and dependency management. The application must use **SQLite as its persistent database**. The selected application domain is flexible and may include areas such as inventory management, supershop POS system, appointment or reservation systems, personal finance, laboratory resource management, task and project management, document workflows, student activity management, simulation applications, developer tools, or other suitable domains. Teams are encouraged to select a problem that is interesting and sufficiently challenging from a software design perspective. However, the project should not be selected solely based on the size of the application or the number of features. The primary consideration should be whether the problem provides meaningful opportunities to demonstrate good software design and the practical use of design patterns.

When selecting a project, teams should identify a **clear problem, target users, core entities, and meaningful application workflows**. The project should not consist primarily of simple CRUD operations. It should include business rules and workflows that require appropriate application logic. As a general guideline, a project should contain a minimum of **4 meaningful database entities/tables**, CRUD functionality for at least three important entities, at least **two meaningful multi-step workflows**, and at least **two meaningful search, analytical, or reporting operations**. The application would normally be expected to contain approximately **4–8 major screens**. These figures are intended to provide guidance for maintaining a reasonable and comparable workload across teams; students should not artificially add entities, screens, or features merely to satisfy numerical targets.

SQLite must be an integral part of the application rather than an additional component added only to satisfy the course requirement. Application data must be stored persistently and remain available after the application is closed and restarted. The database should have a meaningful schema with appropriate primary keys, foreign keys, relationships, constraints, and queries. The application should demonstrate proper interaction between the user interface, application/business logic, and persistence layer. Teams should also consider appropriate separation of responsibilities between different components of the application. Teams are expected to make **seeders** to create tables and load data.

A central objective of the project is the **meaningful application of design patterns**. There is **no fixed minimum or maximum number of design patterns** that every project must use. Instead, each team should identify and apply **as many patterns as are genuinely required by the design of its application**. A pattern should be introduced only when it addresses a real design problem or provides a clear architectural or maintainability benefit. Students must avoid forcing patterns into the application merely to increase the number of patterns demonstrated. For example, creating an unnecessary Singleton, Factory, or Observer solely for the purpose of claiming that the pattern has been used will not be considered good design.

For every significant design pattern used in the project, the team should be able to clearly explain **the problem that led to the pattern, why the selected pattern is appropriate, how it improves the current design, and what alternative approaches were considered**. Students should also discuss the **future benefits** of their design decisions. In particular, teams should explain how the selected design could make the application easier to extend, modify, test, maintain, or integrate with new requirements in the future. For example, if the Strategy pattern is used to support multiple algorithms, students should be able to explain how the design would simplify the addition of a new algorithm later without substantially modifying existing code. The objective is to demonstrate an understanding that

design patterns are tools for managing change and complexity, rather than templates that must be included simply because they are part of the course syllabus.

The project should contain at least one or more **non-trivial workflows** where design decisions and patterns can be demonstrated naturally. Examples may include different pricing or processing strategies, multiple notification mechanisms, state transitions, approval workflows, configurable algorithms, object creation requirements, or interactions between multiple components. The selected workflows should contain sufficient business logic to demonstrate that the application is more than a collection of database forms.

To avoid duplication of project topics between teams, a **shared project-tracking spreadsheet** must be maintained for internal course purposes. Each team should enter its proposed project title, short description, team members, and primary application domain in the designated spreadsheet. Students should check the spreadsheet before selecting a project and should avoid selecting a project that is substantially similar to one already registered by another team. The purpose of the spreadsheet is to maintain diversity among projects and to help coordinate project selection; it is not intended to function as a formal project proposal or approval process. Teams are expected to make reasonable efforts to select sufficiently distinct problems and application domains.

All teams must maintain their project in a **GitHub repository** throughout the development process. The repository must contain the complete source code, Maven configuration, documentation, database-related resources, and other relevant project artifacts. Both members of the team must make meaningful contributions to the repository, and individual contributions will be evaluated through the project's Git history. Students should not combine all work into a small number of commits or rely on one member to perform the majority of the development. Each student's contribution should be clearly identifiable through commits, branches, pull requests, code changes, documentation, testing, or other relevant project activities.

Teams are required to follow **proper Git branching and merging practices**. Development of each significant feature or task should be performed on an appropriate feature branch rather than directly on the main branch. Once a feature has been completed and tested, it should be merged into the appropriate integration branch using a proper pull request or equivalent review process. Teams should maintain a clean and understandable repository history and should demonstrate appropriate practices for handling merges and resolving conflicts. The purpose of this requirement is not merely to use GitHub for storing source code, but to give students practical experience with collaborative software development and version-control workflows used in professional software projects.

The project must be implemented using **Maven for dependency management and project configuration**. Teams should maintain an appropriate Maven project structure and declare required dependencies through the Maven configuration rather than manually adding external libraries to the project. The project should be organized in a manner that supports maintainability and future development. Appropriate separation between the JavaFX presentation layer, application/business logic, design-pattern-related components, and database/persistence components should be considered wherever applicable.

The final project should include a working JavaFX application, SQLite database, complete Maven project configuration, source code, UML class diagrams relevant to the major design decisions, a database ER diagram, and appropriate documentation. The documentation should explain the major architectural decisions, the design patterns used, the problems addressed by those patterns, and the expected future benefits of the selected designs. Teams should also provide appropriate test cases for important business logic and demonstrate that the application handles common validation and error conditions.

During the final demonstration, each team must demonstrate the major workflows of the application and explain the important design decisions. Students may be asked to identify the implementation of specific design patterns in the source code, explain why a particular pattern was selected, discuss alternatives, and describe how the design could accommodate future changes. Each student should be capable of explaining the portions of the system they contributed to and the relevant design decisions.